

Minimum Spanning Tree

2026-1학기 부경대학교 알고리즘 스터디

목차

- Union Find

- 자료구조
- disjoint-set forests
- 구현
- 응용

- MST(최소 신장 트리)

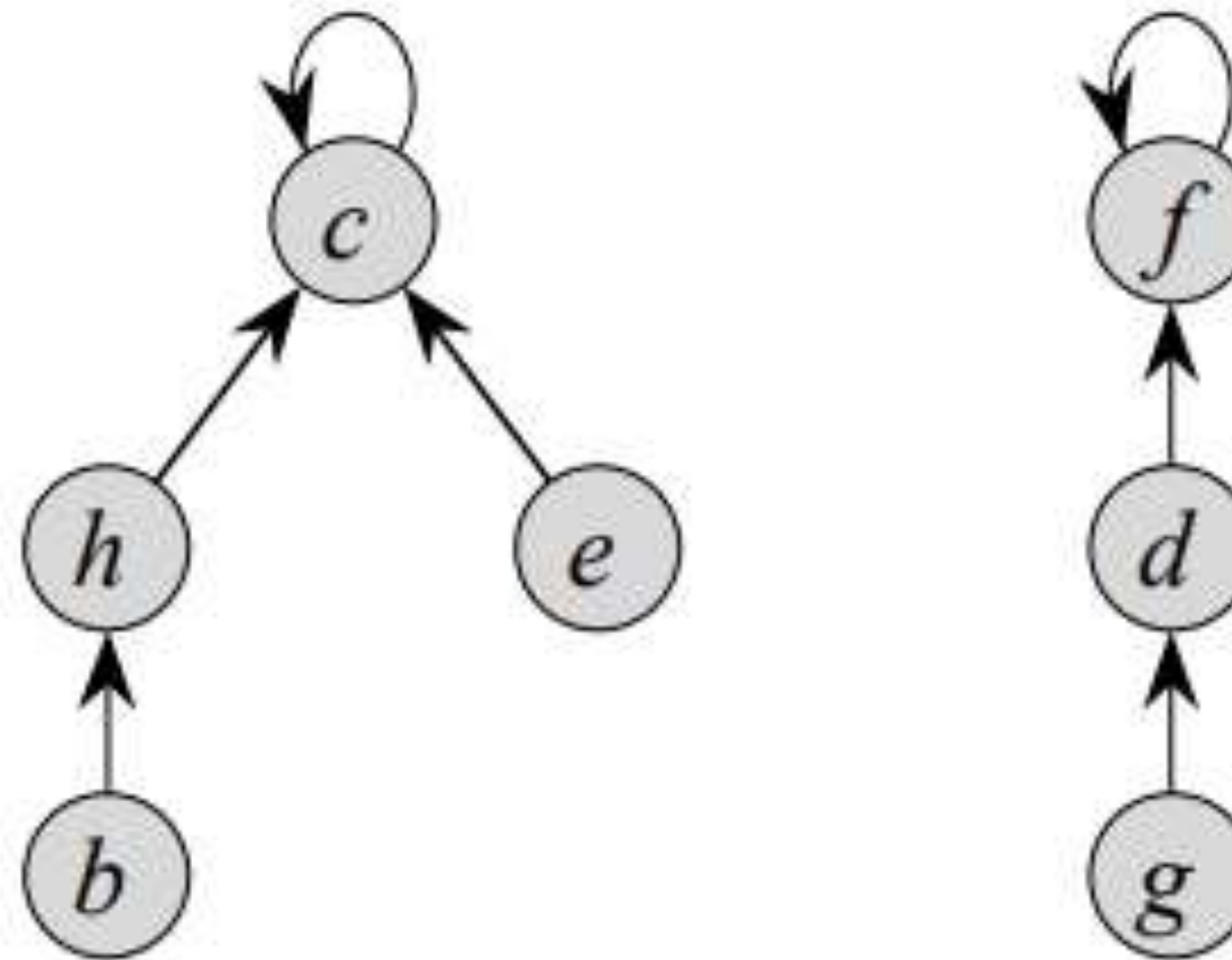
- Kruskal
- Prim

자료구조

- 서로 겹치지 않는 집합들의 모음을 저장
- ex) $\{\{1, 3\}, \{2, 4\}\}$

disjoint-set forests

- 집합을 트리로 표현
 - 집합을 대표하는 집합 내의 원소(root)
 - 다른 원소들은 집합을 대표하는 원소를 조상으로 가지는 트리에 노드로 포함
 - $p[]$: 현재 노드의 부모 노드를 가리키는 배열
- $\{c, h, b, e\}, \{f, d, g\}$ 을 그린 그림



disjoint-set forests

- 연산

- Union(x, y): x 원소가 포함된 트리의 대표 원소의 부모를 y 원소가 포함된 트리의 대표 원소로 변경
- Find(x): x 가 포함된 트리의 root를 반환

- 시간복잡도

- 2개의 연산 모두 $O(1)$ 은 아니나 사실 상 $O(1)$ 이라 봐도 무방함
 - ※ Find의 최악의 경우 시간복잡도는 $O(n)$ 이나 p 배열 초기화 과정에서 $O(n)$ 의 시간이 걸려 상관이 없음
 - ※ 위의 최악의 경우를 방지하기 위해 Union에서 union by rank, union by size가 존재함
 - ※ 이 때의 시간복잡도는 최악의 경우 $O(\alpha(n))$ 이 됨

구현

```
vector<int> p;

int Find(int x) {
    if (p[x] == x) return x;
    else return p[x] = Find(p[x]);
}

void Union(int a, int b) {
    int pa = Find(a), pb = Find(b);

    if (pa != pb) {
        p[pa] = pb;
    }
}
```


응용

- size 저장: $size[]$ = 해당 원소를 대표 원소로 두는 집합의 크기
 - 집합의 크기 구하기
- 배열 저장: $arr[][]$ = 해당 원소를 대표원소로 두는 집합이 가지는 배열
 - Union 시간복잡도
 - 원소의 개수가 n 개이고 집합이 가지는 배열의 총 길이가 n 일 때 단순히 합친다면 $O(n^2)$
 - 배열의 크기가 작은 쪽을 큰 쪽에 합친다면 $O(n \log(n))$

종교 JUNGOL 1863

- 문제

- $n(0 < n \leq 50,000)$ 명의 학생이 있고 각 학생의 번호는 1번 이상 n 번 이하이다.
- $m(0 \leq m \leq 100,000)$ 개의 관계가 주어진다. 각 관계는 $a\ b$ 로 주어지며 a 와 b 가 같은 종교라는 것을 나타낸다.
- 이 학교에 있는 종교의 최대 가지 수를 출력한다.

종교 JUNGOL 1863

- 문제

- $n(0 < n \leq 50,000)$ 명의 학생이 있고 각 학생의 번호는 1번 이상 n 번 이하이다.
- $m(0 \leq m \leq 100,000)$ 개의 관계가 주어진다. 각 관계는 $a\ b$ 로 주어지며 a 와 b 가 같은 종교라는 것을 나타낸다.
- 이 학교에 있는 종교의 최대 가지 수를 출력한다.

- 풀이

- Union으로 관계가 있는 학생을 합침
- 마지막에 $p[x] == x$ 인 것의 수를 세면 현재 집합의 개수를 셀 수 있고 그 집합의 개수가 종교의 최대 가지 수가 됨
- 시간복잡도는 $O(n+m)$

종교 JUNGOL 1863

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> p;

int Find(int x) {
    if (p[x] == x) return x;
    else return p[x] = Find(p[x]);
}

void Union(int a, int b) {
    int pa = Find(a), pb = Find(b);

    if (pa != pb) {
        p[pa] = pb;
    }
}
```

```
int main() {
    int n, m;

    cin >> n >> m;

    p.resize(n+1);
    for (int i = 1; i <= n; i++) p[i] = i;
    while (m--) {
        int a, b;
        cin >> a >> b;
        Union(a, b);
    }

    int res = 0;
    for (int i = 1; i <= n; i++) {
        if (p[i] == i) res++;
    }

    cout << res << "\n";
}
```

MST

- 정의: 연결된 가중 간선 무방향 그래프의 간선 부분집합으로, 사이클 없이 가능한 최소 총 간선 가중치를 가지며 모든 정점을 연결함
- 같은 그래프에서 여러 개의 MST가 나올 수 있음
- MST를 만드는 알고리즘
 - Kruskal: 정렬을 사용, 간선 기준
 - Prim: 우선순위 큐를 사용, 정점 기준

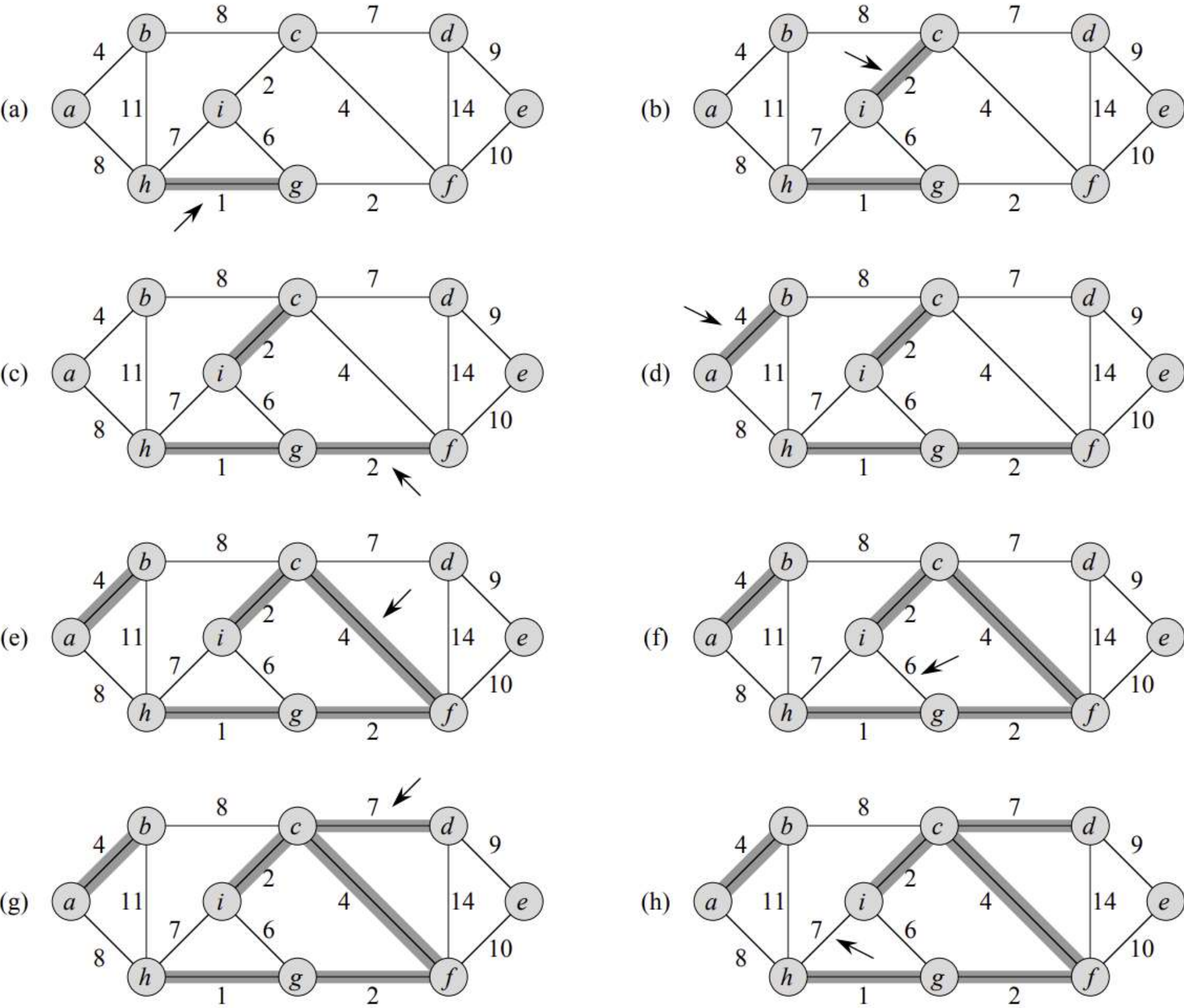
Kruskal

- 알고리즘

- 가중치가 최소인 간선 선택
- 간선과 연결된 정점이 연결되어 있지 않다면 해당 간선 선택
- 모든 간선을 보지 않았다면 1번으로 돌아감

- 구현

- 가중치가 가장 작은 간선을 찾기 쉽게 간선의 가중치를 기준으로 정렬을 함
- 연결되었는지 확인을 위해 Union Find 사용
- 시간복잡도는 $O(E \log(E))$



Kruskal 과정의 일부

Kruskal - 구현

```
struct Edge {  
    int u, v, d;  
  
    Edge(int _u, int _v, int _d)  
        : u(_u), v(_v), d(_d) {}  
};  
  
vector<Edge> edges;  
  
int Kruskal() {  
    int res = 0;  
  
    sort(edges.begin(), edges.end(),  
        [](const Edge& a, const Edge& b) {  
            return a.d < b.d;  
        });  
  
    for (auto& e : edges) {  
        if (Find(e.v) != Find(e.u)) {  
            res += e.d;  
            Union(e.v, e.u);  
        }  
    }  
  
    return res;  
}
```

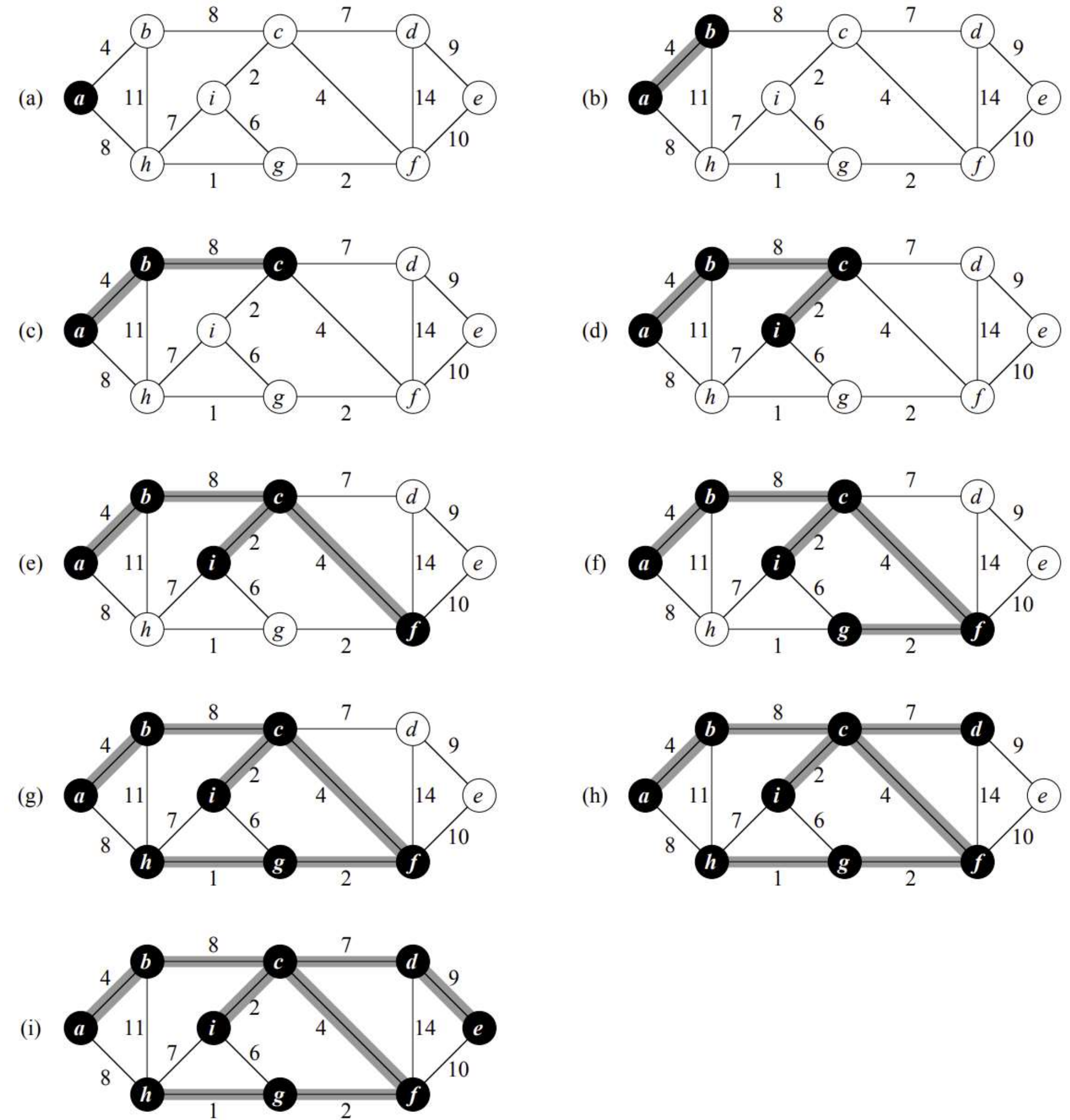

Prim

• 알고리즘

1. 임의의 정점을 선택 후 집합에 넣음
2. 해당 집합과 연결되면서 가중치가 가장 작은 간선과 연결된 정점을 집합에 넣음
3. 모든 정점이 집합에 들어가면 종료

• 구현

- 가중치가 가장 작은 간선을 우선순위 큐로 찾음
- 정점이 집합에 들어갔는지 알기 위해 visited 배열을 사용
- 시간복잡도는 $O(E \log(E))$



Prim 과정의 일부

Prim - 구현

```
using pii = pair<int, int>; // 가중치, 정점
....
vector<vector<pii>> adj;

int Prim() {
    int res = 0;
    priority_queue<pii,
        vector<pii>,
        greater<pii>> pq;
    vector<bool> visited(adj.size());

    pq.push(make_pair(0, 0));
    while (pq.size()) {
        auto t = pq.top(); pq.pop();
        ....
        if (visited[t.second]) continue;
        visited[t.second] = true;
        res += t.first;

        for (int nxt : adj[t.second]) {
            if (!visited[nxt.second]) {
                pq.push(nxt);
            }
        }
    }

    return res;
}
```

최소비용신장트리 JUNGOL 1060

- 문제
 - n ($3 \leq N \leq 100$)개의 학원이 있을 때 $\text{mat}[][]$ (n by n 행렬, 100,000이하)이 주어진다. $\text{mat}[i][j]$ 는 i 번 학원에서 j 번 학원을 연결할 때 드는 비용이다.
 - 모든 학원을 연결하기 위한 최소 비용을 출력한다.

최소비용신장트리 JUNGOL 1060

- 문제
 - n ($3 \leq N \leq 100$)개의 학원이 있을 때 $mat[][]$ (n by n 행렬, 100,000이하)이 주어진다. $mat[i][j]$ 는 i 번 학원에서 j 번 학원을 연결할 때 드는 비용이다.
 - 모든 학원을 연결하기 위한 최소 비용을 출력한다.
- 풀이
 - 크루스칼이나 프림을 사용해서 mst를 만들면서 사용한 간선의 비용을 모두 더한다.
 - 시간복잡도는 $O(n^2 \log(n))$ 이다.

최소비용신장트리 JUNGOL 1060

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

vector<int> p;

int Find(int x) {
    if (p[x] == x) return x;
    else return p[x] = Find(p[x]);
}

void Union(int a, int b) {
    int pa = Find(a), pb = Find(b);

    if (pa != pb) {
        p[pa] = pb;
    }
}
```

```
struct Edge {
    int u, v, d;

    Edge(int _u, int _v, int _d)
        : u(_u), v(_v), d(_d) {}
};

vector<Edge> edges;

int Kruskal() {
    int res = 0;

    sort(edges.begin(), edges.end(),
        [](const Edge& a, const Edge& b) {
            return a.d < b.d;
        });

    for (auto& e : edges) {
        if (Find(e.v) != Find(e.u)) {
            res += e.d;
            Union(e.v, e.u);
        }
    }

    return res;
}
```

```
int main() {
    int n;
    cin >> n;
    p.resize(n);
    for (int i = 0; i < n; i++) {
        p[i] = i;
        for (int j = 0; j < n; j++) {
            int d;
            cin >> d;
            if (i != j)
                edges.emplace_back(i, j, d);
        }
    }

    cout << Kruskal();
}
```

연습문제

- [8406](#) 종교2
- [5187](#) 워터링 홀(Watering Hole)
- [6154](#) 행성 터널
- [2534](#) The Gangs
- [3431](#) 개구리 점프
- [8681](#) 벌집 네트워크